

PROJECT STARK

Reproducible Local Training Infrastructure for Stark Founder V0

M6 Technical Research Report

Project Stark Research Technical Paper

9 September 2026

Accepted training-infrastructure milestone; companion to Founder V0 baseline research

Public project technical research paper. Not externally peer-reviewed.

Abstract

M6 transformed the Stark Founder V0 architecture from a trainable language-model definition into a reproducible local training system. The milestone introduced source-aware token shards, exact and near-overlap leakage controls, deterministic packing, sequence-length curriculum, CUDA/BF16 training, atomic checkpointing, optimizer/scaler/RNG/data-stream restoration, finite-gradient safeguards, PAD invariants, and a fail-conservative continual-learning boundary. Operator testing exposed and corrected several defects, including a broken synthetic-stream RNG checkpoint path, a memorization gate that did not actually train on the probe sentence, and a RoPE cache that expanded exponentially across BF16/FP32 precision transitions. The corrected M6 system subsequently supported PRETRAIN-001 and the frozen Founder V0 baseline.

M6 is the only M1-M6 construction milestone that warrants a separate technical research report. M1-M5 are conventional architectural assembly stages and are more coherently documented together in the Founder V0 architecture/baseline report.

1. Scope

M6 added a reusable training engine around the frozen M5 Founder LM without changing M0-M5 parameter shapes or model architecture.

The system had four principal goals:

1. preserve dataset and tokenizer identity;
2. produce deterministic and crash-resumable training trajectories;
3. support the target local CUDA hardware safely;
4. define a boundary between transient/episodic knowledge and slow parametric consolidation.

2. Data pipeline

The M6 data system implemented:

- source-aware JSONL/text ingestion;
- frozen-tokenizer hash verification;
- BOS/EOS document boundaries;
- trusted user/assistant role insertion only when source metadata justified it;
- reserved control-token rejection;
- uint16 token shards and document indexes;
- SHA-256 manifests;
- exact within-split deduplication;
- cross-split exact leakage rejection;
- conservative token-sketch near-overlap filtering from evaluation against train/replay;
- deterministic weighted packing;
- long-document continuation across physical windows;
- exact stream checkpoint/resume.

These controls are important because model reproducibility is not meaningful if data selection or stream position can drift silently after restart.

3. Trainer

The accepted trainer used AdamW, token-based warmup plus cosine decay, gradient clipping, microbatching and gradient accumulation, and a sequence curriculum:

64 -> 128 -> 256 -> 512 -> 1024

Precision policy supported FP32, FP16 with GradScaler, and BF16. The accepted local accelerator supported BF16 execution.

M6 also implemented validation, early stopping, throughput/VRAM metrics, embedding/attention probes, generation probes, finite loss/gradient failure paths, and exact PAD protection.

4. Exact restart as an experimental requirement

The checkpoint boundary included:

- model Safetensors;
- optimizer state;
- AMP scaler state;
- CPU/CUDA RNG state;
- exact data-stream position;
- tokenizer and dataset identities.

The corrected RNG checkpoint format serializes and verifies the full random-state payload before restoration, then performs a read-back check to confirm the restored accelerator RNG state.

The M6 resume repairs were driven by operator-discovered defects rather than hidden post-hoc. The defects included incorrectly labeled checkpoint progress, reuse of live model/optimizer objects rather than a true restart boundary, a synthetic stream whose `state_dict()` called `seed()` instead of `getstate()`, and a Safetensors comparator that used the wrong loader. The corrected M6.2B equivalence experiment established bitwise trajectory preservation.

5. True-overfit correction

The original memorization gate did not guarantee that its probe string was in the training data and stopped on a weak loss threshold. M6.2 was corrected to use a dedicated synthetic corpus containing a deliberately unique control sequence.

The strong gate required deterministic evaluation loss below 0.25 and an exact greedy continuation match. Sampling remained diagnostic only. This changed the question from “does the loss become smaller?” to “can the training engine intentionally force the model to learn an exact controlled sequence?”

6. RoPE precision-transition cache failure

A particularly useful M6 failure occurred during alternating BF16 training and FP32 generation probes. The RoPE cache treated a dtype mismatch as a reason for geometric growth, causing the local cache capacity to grow:

```
1024 -> 2048 -> 4096 -> ... -> 524288
```

Despite a physical model window of 1,024 tokens, the eight layers eventually accumulated approximately GiB-scale trigonometric cache storage and produced a CUDA OOM.

The fix restricted geometric cache growth to genuine same-device/same-dtype sequence-length growth. A precision/device transition now rebuilds only at `max(required_positions, initial_cache_length)`. A regression test repeatedly alternates BF16/FP32 and requires cached positions to remain 1,024.

This incident is important because it validates the project's bounded-context invariant under real mixed-precision execution, not only in static architecture review.

7. Continual-learning boundary

M6 did not equate every new experience with a gradient update. It defined a stability taxonomy and a conservative bridge in which dynamic and episodic information remain in fast/external memory, while only sufficiently stable and verified material can enter a slow weight-consolidation queue. A replay mixture is required to reduce catastrophic forgetting risk; the initial default replay ratio is 20%.

This separation later became conceptually consistent with M8's stronger distinction between memory state, exact evidence, and consolidation.

8. Why M6 deserves a separate report

M1-M5 are essential engineering milestones but do not independently establish new empirical claims beyond correct construction of a conventional small Transformer substrate. M6, by contrast, produced several independently falsifiable systems results:

- reproducible local CUDA/BF16 training;
- exact checkpoint/resume semantics;
- controlled intentional memorization;
- real-data generalization infrastructure;
- dataset leakage controls;
- operator-discovered precision/cache failure and correction;
- a governed separation between fast memory and slow parametric consolidation.

The successful PRETRAIN-001 run and M7 freeze depend directly on these guarantees.

9. Conclusion

M6 should be treated as a technical infrastructure research milestone rather than just a trainer implementation. Its main contribution is not a novel optimizer or model block. It is the establishment of an auditable local experimental substrate in which interruption, precision transitions, dataset identity, RNG state, and later consolidation can be tested rather than assumed.

For public documentation, M1-M5 remain best represented as a single architecture-construction sequence in the Founder V0 paper. M6 merits a standalone technical report because it carries reproducibility, failure-analysis, and continual-learning implications that persist across every later Stark experiment.

Primary Project Evidence

The report is based on accepted M6 architecture, implementation, correction, and acceptance evidence, together with the later pretraining and Founder V0 baseline records that depended on those guarantees. The internal archive retains exact artifact identities, operational filenames, and machine-specific evidence; those details are intentionally omitted from this public edition.

This document is a project technical report, not a claim of external peer review.